

Week-04 HandsOn Solution

❖ JWT Microservices

Question 1: Implement JWT Authentication in ASP.NET Core Web API

- Code:

```
using System;

using JwtMicroservice.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace JwtMicroservice.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        [HttpPost("login")]
        public IActionResult Login([FromBody] LoginModel model)
        {
            if (!IsValidUser(model))
            {
                return Unauthorized();
            }

            var token = GenerateJwtToken(model.Username);
            return Ok(new { Token = token });
        }

        private bool IsValidUser(LoginModel model)
        {
            return model.Username == "admin" && model.Password == "password";
        }

        private string GenerateJwtToken(string username)
        {
            var claims = new[]
            {
                new Claim(ClaimTypes.Name, username),
                new Claim(ClaimTypes.Role, "Admin")
            };

            var key = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes("ThisIsASecretKeyForJwtToken"));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: "MyAuthServer",
                audience: "MyApiUsers",
                claims: claims,
                expires: DateTime.Now.AddMinutes(60),
                signingCredentials: creds);

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

```

    }
}
}

```

- **Output:**

Responses		
Curl	<pre>curl -X 'POST' \ 'https://localhost:5001/api/auth/login' \ -H 'accept: */*' \ -H 'Content-Type: application/json' \ -d '{ "username": "admin", "password": "password" }'</pre>	
Request URL	https://localhost:5001/api/auth/login	
Server response		
Code	Details	
200	Response body	<pre>{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpvcC9yZWVudmVhc2NoZmlkcy54MmZrb2FudW9yZy9ycyBMDA11zA112lXN2b2xBSUJ2bWY1Ijo1YWRtaW11ClodHw0IHRlc2NoZmlkcy5tYWlyb3NvZnQvZj91L3drl1FuODRvMDFvaW1mZDpkbnV2Z2haak1zL3ZvbG01OjB7Gpb11sInV4CiE1ONTClmZyODE0MS1laXoz1jo1TXIBdXRuZ2VydWVy1iw1YXN1L3Jo1TXIBGV2Yycy39-VPTeH3Qw4HLN06dw3npr4kw_IFNvaz1OC7EXSyS8RW"</pre> Download
	Response headers	<pre>content-type: application/json; charset=utf-8 date: Sun, 20 Jul 2025 15:15:42 GMT server: Kestrel</pre>
Responses		
Code	Description	Links
200	Success	No links

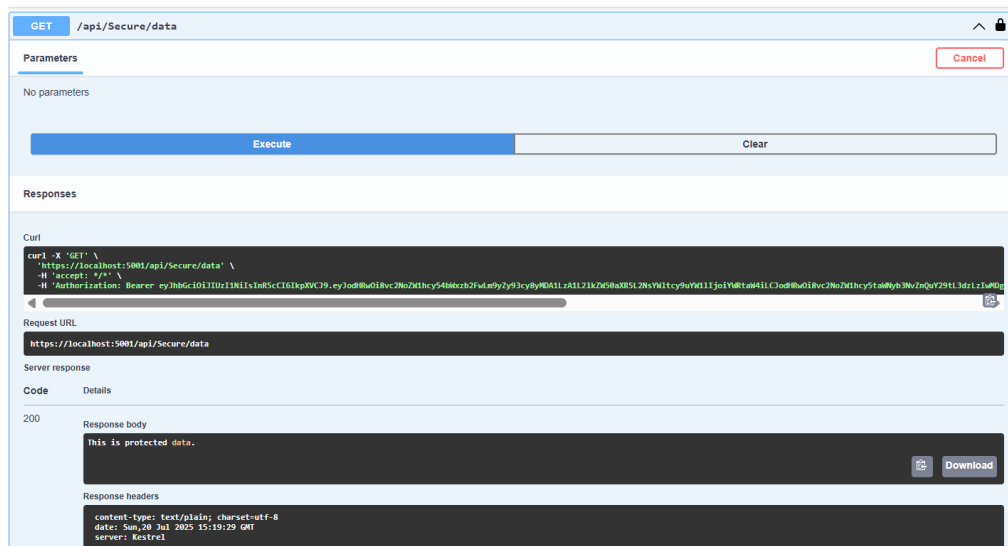
Question 2: Secure an API Endpoint Using JWT

- **Code:**

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace JwtMicroservice.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class SecureController : ControllerBase
    {
        [HttpGet("data")]
        [Authorize]
        public IActionResult GetSecureData()
        {
            return Ok("This is protected data.");
        }
    }
}
```

- Output:



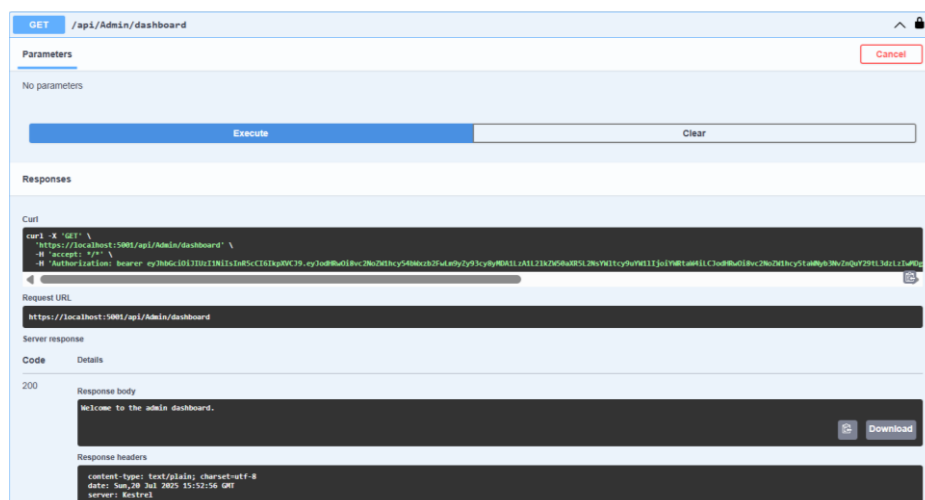
Question 3: Add Role-Based Authorization

- **Code:**

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc;

namespace JwtMicroservice.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AdminController : ControllerBase
    {
        [HttpGet("dashboard")]
        [Authorize(Roles = "Admin")]
        public IActionResult GetAdminDashboard()
        {
            return Ok("Welcome to the admin dashboard.");
        }
    }
}
```

- **Output:**



Question 4: Validate JWT Token Expiry and Handle Unauthorized Access

- **Code:**

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using System.Text;
using System.Threading.Tasks;
using System;

var builder = Microsoft.AspNetCore.Builder.WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        };

        options.Events = new JwtBearerEvents
        {
            OnAuthenticationFailed = context =>
            {
                if (context.Exception is SecurityTokenExpiredException)
                {
                    context.Response.Headers.Append("Token-Expired", "true");
                }
                return Task.CompletedTask;
            }
        };
    });

builder.Services.AddAuthorization();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "JWT API", Version = "v1" });

    c.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Description = "JWT Authorization header using the Bearer scheme.",
        Name = "Authorization",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer"
    });

    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme {
                Reference = new OpenApiReference {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

```
var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```